THỰC HÀNH 5: XỬ LÝ LỖI & UNIT TESTING (EXCEPTIONS & TESTING)

🎯 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài thực hành này, bạn sẽ:

1. ✅ **Phân biệt** Checked vs Unchecked Exceptions
2. ✅ **Xử lý Exception** đúng cách với try-catch-finally
3. ✅ **Sử dụng** Try-with-resources để quản lý tài nguyên
4. ✅ **Tạo Custom Exception** cho business logic
5. ✅ **Viết Unit Test** với JUnit 5
6. ✅ **Áp dụng** Testing best practices (AAA Pattern)

⏱️ **Thời gian ước tính:** 6-8 giờ

📋 **Kiến thức cần có trước:**

- Đã hoàn thành Thực hành 4
- Đã đọc Chương 5: Exceptions & Testing
- Hiểu về Class, Method

🧪 PHẦN 1: HƯỚNG DẪN CHI TIẾT (STEP-BY-STEP)

LAB 5.1: Máy Tính "Bất Tử" (Robust Calculator)

Mục tiêu: Xử lý mọi input rác mà không crash

Bước 1: Phân Tích Vấn Đề

Vấn đề:

- User có thể nhập: "abc" (chữ), "0" (chia cho 0), "@#$" (ký tự lạ)
- App không được Crash
- Phải thông báo lịch sự và cho nhập lại

Bước 2: Viết Code với Exception Handling

Tạo file: `SecureCalculator.java`

```
package com.bootcamp.lab5;

import java.util.InputMismatchException;
import java.util.Scanner;

public class SecureCalculator {
```

```java
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== MÁY TÍNH BẤT TỬ ===\n");

        while (true) {
            try {
                // Vùng nguy hiểm (Risky Zone)
                System.out.print("Nhập số bị chia: ");
                int a = sc.nextInt();

                System.out.print("Nhập số chia: ");
                int b = sc.nextInt();

                // Logic có thể gây lỗi toán học
                int result = divide(a, b);

                System.out.println("✅ Kết quả: " + result);
                break;  // Thành công → Thoát vòng lặp

            } catch (InputMismatchException e) {
                // Xử lý lỗi nhập sai kiểu (Chữ thay vì số)
                System.out.println("❌ Lỗi: Bạn nhập chữ cái! Vui lòng
nhập số.");
                sc.nextLine();  // QUAN TRỌNG: Nuốt bỏ dòng nhập sai

            } catch (ArithmeticException e) {
                // Xử lý lỗi toán học (Chia 0)
                System.out.println("❌ Lỗi toán: " + e.getMessage());

            } catch (Exception e) {
                // Lưới bắt mọi loại lỗi không ngờ tới
                System.out.println("❌ Lỗi hệ thống: " + e.toString());
                e.printStackTrace();  // In stack trace để debug
            }
        }

        sc.close();
    }

    // Method có thể ném exception
    public static int divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Cấm chia cho số 0!");
        }
        return a / b;
    }
}
```

Giải thích:

1. **Try-Catch:**

    - `try`: Vùng code có thể lỗi
    - `catch`: Xử lý từng loại lỗi cụ thể
    - Thứ tự: Từ cụ thể → chung chung

2. **`sc.nextLine()` trong catch:**
    - Khi `nextInt()` lỗi, dòng nhập sai vẫn còn trong buffer
    - `nextLine()` nuốt bỏ dòng đó
    - Nếu không có → Vòng lặp vô tận

3. **`throw`:**
    - Ném exception tùy chỉnh
    - `ArithmeticException`: Unchecked exception

Bước 3: Test Các Trường Hợp

Test case 1: Nhập chữ

```
Nhập số bị chia: abc
❌ Lỗi: Bạn nhập chữ cái! Vui lòng nhập số.
```

Test case 2: Chia cho 0

```
Nhập số bị chia: 10
Nhập số chia: 0
❌ Lỗi toán: Cấm chia cho số 0!
```

Test case 3: Nhập đúng

```
Nhập số bị chia: 10
Nhập số chia: 2
✅ Kết quả: 5
```

LAB 5.2: Custom Exception + Unit Test

Mục tiêu: Tạo exception riêng và viết test

Bước 1: Tạo Custom Exception

Tạo file: `InsufficientFundsException.java`

```
package com.bootcamp.lab5;
```

```java
// Custom Exception - Kế thừa từ RuntimeException (Unchecked)
public class InsufficientFundsException extends RuntimeException {
    private double balance;
    private double amount;

    public InsufficientFundsException(double balance, double amount) {
        super("Thiếu tiền! Bạn có " + balance + " VND, nhưng muốn rút " +
amount + " VND");
        this.balance = balance;
        this.amount = amount;
    }

    public double getBalance() {
        return balance;
    }

    public double getAmount() {
        return amount;
    }
}
```

Giải thích:

- `extends RuntimeException`: Unchecked exception (không bắt buộc catch)
- Có thể thêm thông tin riêng (`balance`, `amount`)

Bước 2: Sử Dụng Custom Exception

Tạo file: `BankAccount.java`

```java
package com.bootcamp.lab5;

public class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Số dư ban đầu không được
âm");
        }
        this.balance = balance;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Tiền rút phải dương");
        }

        if (amount > balance) {
            // Ném custom exception
            throw new InsufficientFundsException(balance, amount);
```

```
        }

        this.balance -= amount;
        System.out.println("✅ Đã rút " + amount + " VND. Số dư còn: " +
balance);
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Tiền nạp phải dương");
        }
        this.balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

Bước 3: Viết Unit Test với JUnit 5

Cài đặt JUnit 5 (Maven):

Thêm vào `pom.xml`:

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Tạo file: `BankAccountTest.java` (trong thư mục `src/test/java`)

```
package com.bootcamp.lab5;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class BankAccountTest {
    private BankAccount account;

    // Chạy trước mỗi test
    @BeforeEach
    public void setUp() {
        account = new BankAccount(1000.0);
```

```java
    }

    // Test case: Rút tiền thành công
    @Test
    public void testWithdraw_Success() {
        // Arrange: Đã setup trong @BeforeEach

        // Act: Thực hiện hành động
        account.withdraw(300.0);

        // Assert: Kiểm tra kết quả
        assertEquals(700.0, account.getBalance(), 0.01);
    }

    // Test case: Rút quá số dư → Ném exception
    @Test
    public void testWithdraw_InsufficientFunds() {
        // Act & Assert: Kiểm tra ném exception
        InsufficientFundsException exception = assertThrows(
            InsufficientFundsException.class,
            () -> account.withdraw(2000.0)  // Lambda expression
        );

        // Kiểm tra thông điệp lỗi
        assertTrue(exception.getMessage().contains("Thiếu tiền"));
        assertEquals(1000.0, exception.getBalance(), 0.01);
        assertEquals(2000.0, exception.getAmount(), 0.01);
    }

    // Test case: Rút số âm → Ném IllegalArgumentException
    @Test
    public void testWithdraw_NegativeAmount() {
        IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> account.withdraw(-100.0)
        );

        assertTrue(exception.getMessage().contains("phải dương"));
    }

    // Test case: Nạp tiền thành công
    @Test
    public void testDeposit_Success() {
        account.deposit(500.0);
        assertEquals(1500.0, account.getBalance(), 0.01);
    }

    // Test case: Nạp số âm → Ném exception
    @Test
    public void testDeposit_NegativeAmount() {
        assertThrows(
            IllegalArgumentException.class,
            () -> account.deposit(-100.0)
        );
```

```
    }
}
```

Giải thích:

1. **@BeforeEach:**
   - Chạy trước mỗi test
   - Đảm bảo mỗi test có môi trường sạch
2. **AAA Pattern:**
   - **Arrange**: Chuẩn bị dữ liệu
   - **Act**: Thực hiện hành động
   - **Assert**: Kiểm tra kết quả
3. **assertThrows:**
   - Kiểm tra method có ném exception không
   - Lambda: `() -> account.withdraw(...)`
4. **assertEquals với delta:**
   - `assertEquals(expected, actual, delta)`
   - `delta`: Sai số cho phép (0.01 cho số thực)

Bước 4: Chạy Test

Cách 1: VS Code

- Click nút **Run Test** bên cạnh `@Test`
- Hoặc chuột phải → **Run Test**

Cách 2: Terminal

```
mvn test
```

Kết quả:

```
✅ testWithdraw_Success
✅ testWithdraw_InsufficientFunds
✅ testWithdraw_NegativeAmount
✅ testDeposit_Success
✅ testDeposit_NegativeAmount
```

LAB 5.3: Try-with-Resources (File I/O)

Mục tiêu: Quản lý tài nguyên tự động

Bước 1: Đọc File với Try-with-Resources

Tạo file: `FileReaderDemo.java`

```
package com.bootcamp.lab5;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class FileReaderDemo {
    public static void main(String[] args) {
        Path path = Paths.get("data.txt");

        // Try-with-resources: Tự động close
        try (BufferedReader reader = Files.newBufferedReader(path)) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("❌ Lỗi đọc file: " + e.getMessage());
        }
        // File tự động được close ở đây!
    }
}
```

Giải thích:

- `try (resource)`: Resource tự động close khi ra khỏi block
- Không cần `finally` để close
- Resource phải implement `AutoCloseable`

Bước 2: Ghi File với Try-with-Resources

```
package com.bootcamp.lab5;

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
```

```java
public class FileWriterDemo {
    public static void main(String[] args) {
        Path path = Paths.get("output.txt");

        try (BufferedWriter writer = Files.newBufferedWriter(path)) {
            writer.write("Dòng 1");
            writer.newLine();
            writer.write("Dòng 2");
            writer.newLine();
            writer.write("Dòng 3");
        } catch (IOException e) {
            System.out.println("❌ Lỗi ghi file: " + e.getMessage());
        }
    }
}
```

Bước 3: Multiple Resources

```java
try (
    BufferedReader reader = Files.newBufferedReader(inputPath);
    BufferedWriter writer = Files.newBufferedWriter(outputPath)
) {
    // Đọc và ghi
    String line;
    while ((line = reader.readLine()) != null) {
        writer.write(line);
        writer.newLine();
    }
} catch (IOException e) {
    e.printStackTrace();
}
// Cả 2 resources tự động close (theo thứ tự ngược lại)
```

📝 PHẦN 2: BÀI TẬP CÓ HƯỚNG DẪN

Bài 5.1: Đọc File An Toàn

Đề bài: Viết chương trình đọc file `config.txt`

Hướng Dẫn

```java
package com.bootcamp.lab5;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
```

```java
public class ConfigReader {
    public static void main(String[] args) {
        Path path = Paths.get("config.txt");

        try {
            // Đọc toàn bộ file thành String
            String content = Files.readString(path);
            System.out.println("Nội dung file:");
            System.out.println(content);

        } catch (java.nio.file.NoSuchFileException e) {
            System.out.println("❌ File không tồn tại: " + path);
            System.out.println("💡 Vui lòng tạo file config.txt trước");

        } catch (IOException e) {
            System.out.println("❌ Lỗi đọc file: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Bài 5.2: Kiểm Tra Tuổi với Custom Exception

Đề bài: Nhập năm sinh, tính tuổi. Ném exception nếu không hợp lệ.

Hướng Dẫn

Bước 1: Custom Exception

```java
package com.bootcamp.lab5;

public class InvalidAgeException extends Exception {
    private int age;

    public InvalidAgeException(int age, String message) {
        super(message);
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}
```

Bước 2: Class Validate Age

```java
package com.bootcamp.lab5;

import java.util.Scanner;

public class AgeValidator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nhập năm sinh: ");
        int birthYear = sc.nextInt();

        try {
            int age = calculateAge(birthYear);
            System.out.println("✅ Tuổi của bạn: " + age);

        } catch (InvalidAgeException e) {
            System.out.println("❌ " + e.getMessage());
        }

        sc.close();
    }

    public static int calculateAge(int birthYear) throws
InvalidAgeException {
        int currentYear = 2024;  // Hoặc dùng LocalDate.now().getYear()
        int age = currentYear - birthYear;

        if (age < 18) {
            throw new InvalidAgeException(age, "Bạn chưa đủ tuổi, đi về
đi!");
        }

        if (age > 100) {
            throw new InvalidAgeException(age, "Cụ già quá rồi, nghỉ ngơi
đi");
        }

        return age;
    }
}
```

🎯 PHẦN 3: BÀI TẬP TỰ LUYỆN (KHÔNG CÓ HƯỚNG DẪN)

Bài 5.1: Unit Test cho Calculator

Đề bài: Viết Unit Test cho class `Calculator` với các methods:

- `add(int a, int b)`
- `subtract(int a, int b)`
- `multiply(int a, int b)`

- `divide(int a, int b)` - Ném exception nếu chia 0

Yêu cầu:

- Test các trường hợp bình thường
- Test boundary cases
- Test exception

Bài 5.2: File Manager với Exception Handling

Đề bài: Tạo class `FileManager` với:

- `readFile(String filename)`: Đọc file
- `writeFile(String filename, String content)`: Ghi file
- `appendToFile(String filename, String content)`: Thêm vào cuối

Yêu cầu:

- Sử dụng try-with-resources
- Xử lý tất cả exceptions
- Custom exception `FileManagerException`

Bài 5.3: Validation với Exception

Đề bài: Tạo class `UserValidator`:

- Method `validateEmail(String email)`: Kiểm tra email hợp lệ
- Method `validatePassword(String password)`: Kiểm tra mật khẩu (>=8 ký tự)
- Ném `ValidationException` nếu không hợp lệ

Yêu cầu:

- Custom exception với thông điệp rõ ràng
- Viết Unit Test

Bài 5.4: Age Validator với Custom Exception

Đề bài: Tạo class `AgeValidator`:

- Method `validateAge(int age)`: Kiểm tra tuổi hợp lệ (0-150)
- Ném `InvalidAgeException` nếu:
  - Tuổi < 0: "Tuổi không thể âm"
  - Tuổi > 150: "Tuổi không hợp lệ (quá lớn)"
- Method `getAgeCategory(int age)`: Trả về "Trẻ em", "Thanh niên", "Trung niên", "Cao tuổi"

Gợi ý:

- Tạo custom exception `InvalidAgeException extends Exception`
- Sử dụng try-catch trong main để test

- Viết Unit Test cho các boundary cases

Bài 5.5: Bank Account với Exception Handling

Đề bài: Mở rộng class `BankAccount`:

- Method `withdraw(double amount)`: Ném `InsufficientFundsException` nếu không đủ tiền
- Method `deposit(double amount)`: Ném `InvalidAmountException` nếu amount <= 0
- Method `transfer(BankAccount target, double amount)`: Chuyển tiền, xử lý tất cả exceptions

Gợi ý:

- Tạo 2 custom exceptions
- `transfer()` phải xử lý cả InsufficientFunds và InvalidAmount
- Viết Unit Test cho từng trường hợp

Bài 5.6: File Reader với Multiple Exception Handling

Đề bài: Tạo class `ConfigReader` đọc file config:

- Method `readConfig(String filename)`: Đọc file, parse JSON-like format
- Xử lý các exceptions:
  - `FileNotFoundException`: File không tồn tại
  - `IOException`: Lỗi đọc file
  - `ParseException` (custom): Format file sai
- Trả về `Map<String, String>` chứa config

Gợi ý:

- Dùng try-with-resources cho BufferedReader
- Tạo custom `ParseException extends Exception`
- Parse format đơn giản: `key=value` (mỗi dòng một cặp)

Bài 5.7: Unit Test cho String Utilities

Đề bài: Viết Unit Test cho class `StringUtils`:

- `reverse(String str)`: Đảo ngược chuỗi
- `isPalindrome(String str)`: Kiểm tra đối xứng
- `countWords(String str)`: Đếm số từ
- `capitalize(String str)`: Viết hoa chữ cái đầu

Yêu cầu:

- Test normal cases
- Test edge cases: null, empty string, một ký tự
- Test với `@BeforeEach` để setup test data
- Sử dụng `assertThrows()` cho null input

Gợi ý:

- Tạo test class `StringUtilsTest`
- Dùng `@BeforeEach` để khởi tạo `StringUtils` instance
- Test null → ném `NullPointerException`

Bài 5.8: Robust Calculator với Full Exception Handling

Đề bài: Tạo class `AdvancedCalculator`:

- Methods: `add`, `subtract`, `multiply`, `divide`, `power`, `sqrt`
- Xử lý exceptions:
  - Chia 0 → `ArithmeticException`
  - Căn số âm → `IllegalArgumentException`
  - Lũy thừa quá lớn → `OverflowException` (custom)
- Method `calculate(String expression)`: Parse và tính biểu thức đơn giản

Gợi ý:

- `sqrt()`: Kiểm tra số âm trước khi tính
- `power()`: Kiểm tra overflow (kết quả > Integer.MAX_VALUE)
- `calculate()`: Parse format đơn giản như "5+3", "10/2"
- Viết Unit Test đầy đủ cho tất cả methods

🏆 PHẦN 4: THỬ THÁCH NÂNG CAO

Challenge 1: Shopping Cart với TDD

Đề bài: Làm theo Test-Driven Development (TDD):

Quy trình:

1. Viết Test trước (sẽ fail)
2. Viết code đơn giản nhất để pass test
3. Refactor
4. Lặp lại

Yêu cầu:

1. **Test 1:** `testAddToCart()`
   - Tạo Cart
   - Add item
   - Kiểm tra item có trong cart
2. **Test 2:** `testRemoveItem()`
   - Add item
   - Remove item
   - Kiểm tra item không còn trong cart

3. **Test 3:** `testAddNegativeQuantity()`
    - Thêm item với số lượng âm
    - Ném exception

Mục tiêu: Làm quen với TDD workflow

Challenge 2: Robust File Processor

Đề bài: Tạo class `FileProcessor` xử lý file CSV:

1. **Đọc file CSV:**
    - Format: `name,age,email`
    - Xử lý lỗi: File không tồn tại, format sai
2. **Parse dữ liệu:**
    - Tạo object `Person` từ mỗi dòng
    - Validation: age >= 0, email có @
3. **Ghi file kết quả:**
    - Ghi danh sách người hợp lệ
    - Ghi log lỗi

Yêu cầu:

- Try-with-resources
- Custom exceptions
- Unit Test đầy đủ

📚 BẢNG GHI NHỚ (CHEAT SHEET)

Exception Types

Loại	Kế thừa từ	Phải catch?	Ví dụ
Checked	`Exception`	✅ Có	`IOException`, `FileNotFoundException`
Unchecked	`RuntimeException`	❌ Không	`NullPointerException`, `ArithmeticException`

Try-Catch-Finally

```
try {
    // Code có thể lỗi
} catch (SpecificException e) {
    // Xử lý lỗi cụ thể
```

```
} catch (Exception e) {
    // Xử lý lỗi chung
} finally {
    // Luôn chạy (dù có lỗi hay không)
}
```

Try-with-Resources

```
try (Resource resource = new Resource()) {
    // Sử dụng resource
} catch (Exception e) {
    // Xử lý lỗi
}
// Resource tự động close
```

JUnit 5 Assertions

Assertion	Ý nghĩa
`assertEquals(expected, actual)`	So sánh bằng
`assertTrue(condition)`	Kiểm tra true
`assertNull(object)`	Kiểm tra null
`assertThrows(Exception.class, () -> ...)`	Kiểm tra ném exception

✅ CHECKLIST HOÀN THÀNH

- ☐ Đã phân biệt được Checked vs Unchecked Exceptions
- ☐ Đã xử lý Exception với try-catch-finally
- ☐ Đã sử dụng try-with-resources
- ☐ Đã tạo Custom Exception
- ☐ Đã viết Unit Test với JUnit 5
- ☐ Đã áp dụng AAA Pattern
- ☐ Đã hoàn thành LAB 5.1 (SecureCalculator)
- ☐ Đã hoàn thành LAB 5.2 (BankAccount + Test)
- ☐ Đã hoàn thành LAB 5.3 (File I/O)
- ☐ Đã làm ít nhất 2 bài tập có hướng dẫn
- ☐ Đã làm ít nhất 2 bài tập tự luyện
- ☐ Đã thử ít nhất 1 thử thách nâng cao

🎓 TỔNG KẾT

Sau bài thực hành này, bạn đã:

1. ✅ **Xử lý Exception** đúng cách

2. ✅ **Sử dụng** try-with-resources
3. ✅ **Tạo Custom Exception**
4. ✅ **Viết Unit Test** chuyên nghiệp
5. ✅ **Áp dụng** Testing best practices
6. ✅ **Tư duy phòng thủ** (Defensive Programming)

Bước tiếp theo: Chuyển sang Thực hành 6: Collections & Stream API

Chúc bạn học tốt! 🚀